Variational Autoencoder & Generative Adversarial Network

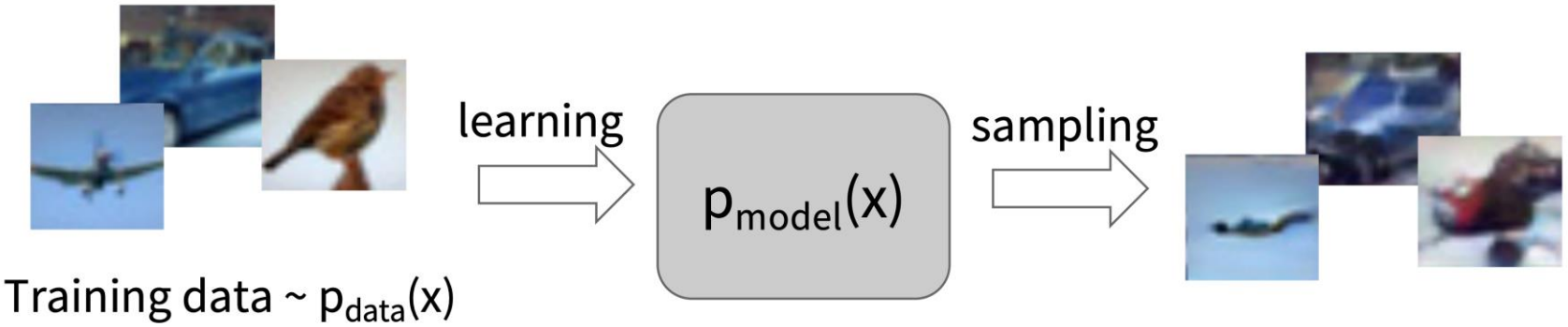
2025. 6. 11.

Jaehoon Lee

Seoul National University
Data Science and AI Laboratory

Generative Models

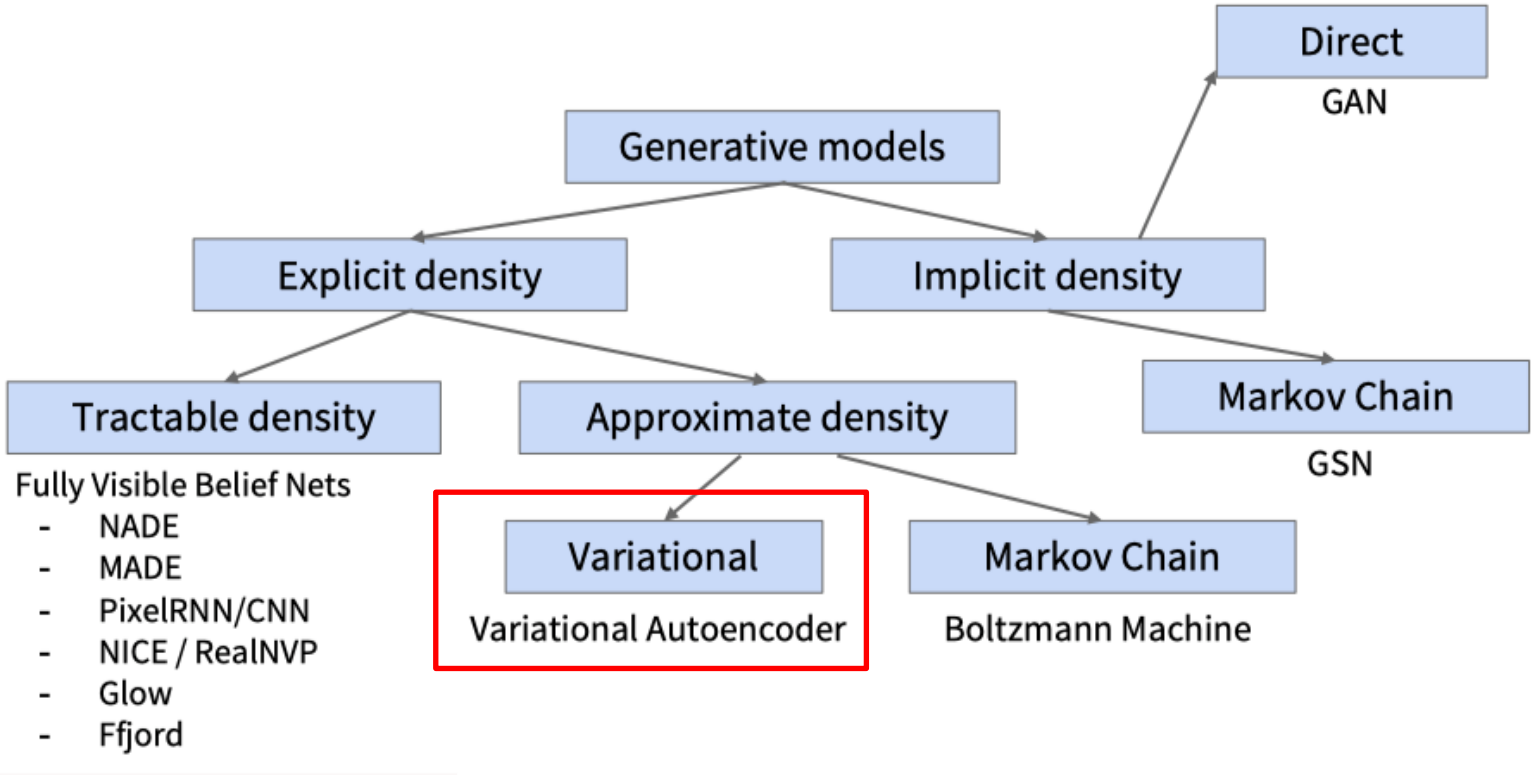
- What are generative models?



  - Training dataset으로부터 data distribution을 학습
  - 학습이 완료된 모델의 distribution으로부터 data sampling

Generative Models

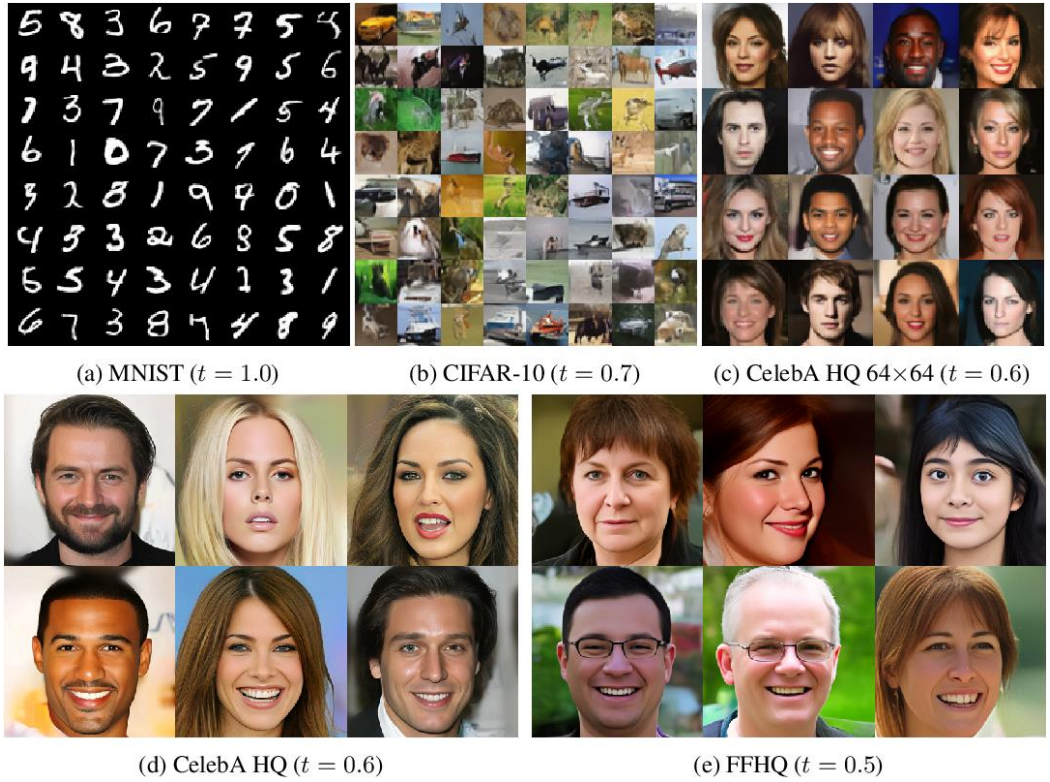
- Taxonomy of Generative models



[1] Ian Goodfewllow, Tutorial on Generative Adversarial Networks, 2017.

Variational Autoencoder

- VAE
  - 확률 모델링을 이용한 생성모델의 전통 강자



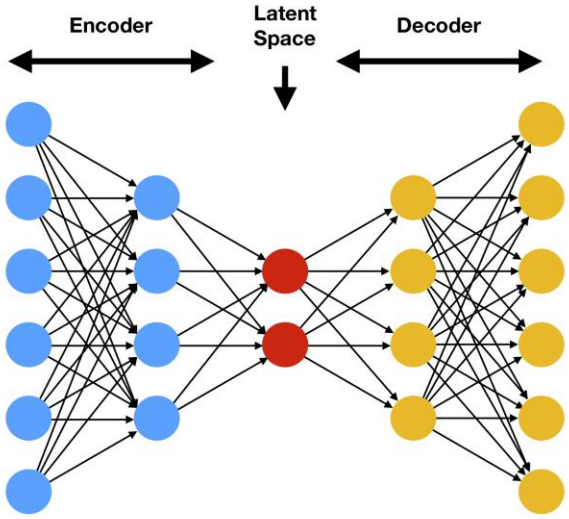
(a) MNIST ($t = 1.0$) (b) CIFAR-10 ($t = 0.7$) (c) CelebA HQ 64×64 ($t = 0.6$)

(d) CelebA HQ ($t = 0.6$) (e) FFHQ ($t = 0.5$)

[1] NVAE: A Deep Hierarchical Variational Autoencoder, Arash et al., 2020

Autoencoder

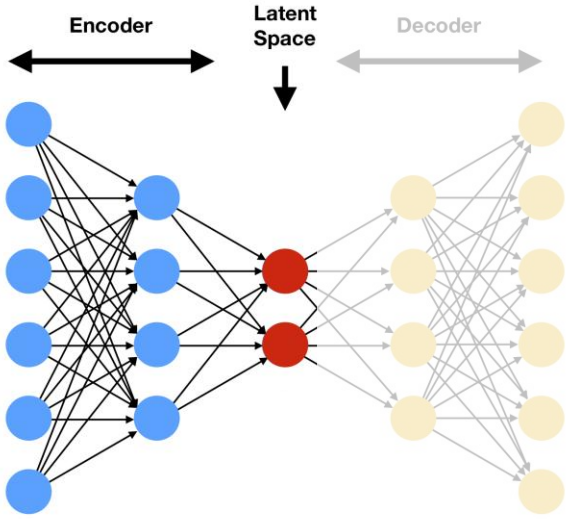
- AE
  - 압축/복원의 과정을 통해 data로부터 저차원의 feature를 unsupervised로 학습하는 방식
  - Encoder
  - Decoder
  - Reconstruction training



[1] https://www.compthree.com/blog/autoencoder/

Autoencoder

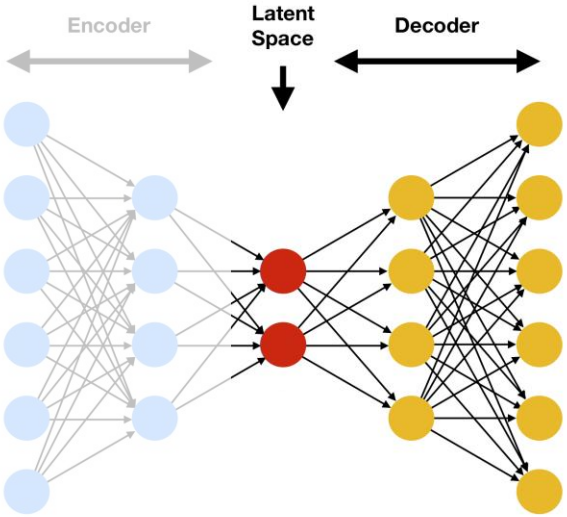
- AE
  - **압축**/복원의 과정을 통해 data로부터 저차원의 feature를 unsupervised로 학습하는 방식
  - Encoder : data로부터 저차원의 feature를 추출
  - Decoder
  - Reconstruction training



[1] https://www.compthree.com/blog/autoencoder/

Autoencoder

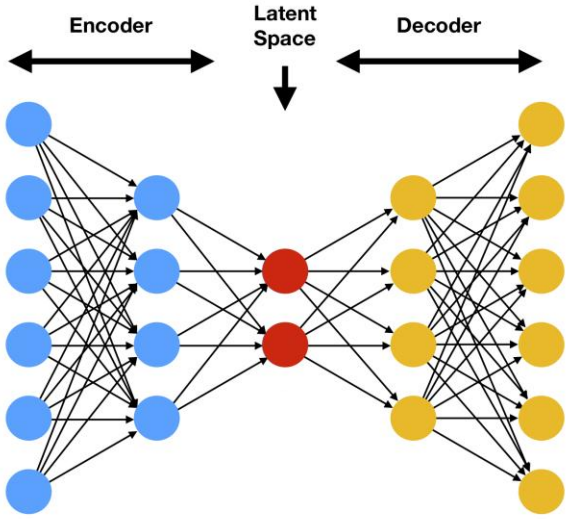
- AE
  - 압축/**복원**의 과정을 통해 data로부터 저차원의 feature를 unsupervised로 학습하는 방식
  - Encoder : data로부터 저차원의 feature를 추출
  - Decoder : 저차원의 feature로부터 다시 data를 복원
  - Reconstruction training



[1] https://www.compthree.com/blog/autoencoder/

Autoencoder

- AE
  - 압축/복원의 과정을 통해 data로부터 저차원의 feature를 **unsupervised로 학습**하는 방식
  - Encoder : data로부터 저차원의 feature를 추출
  - Decoder : 저차원의 feature로부터 다시 data를 복원
  - Reconstruction training $\|x - \hat{x}\|^2$



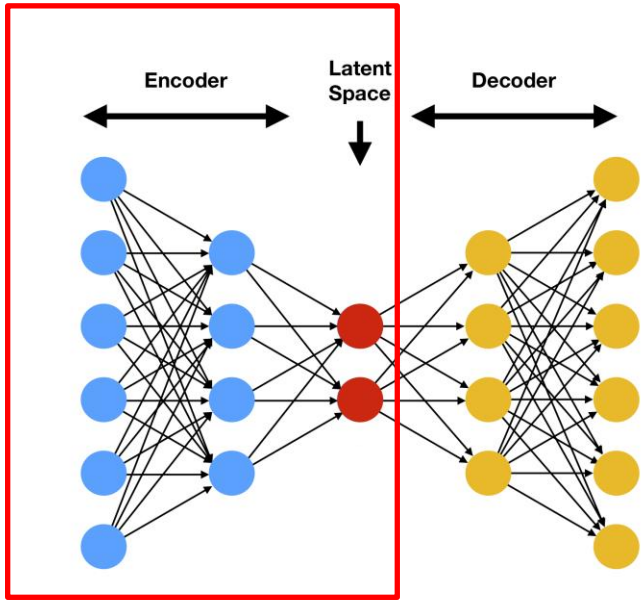
[1] https://www.compthree.com/blog/autoencoder/

Autoencoder

- AE
  - 압축/복원의 과정을 통해 data로부터 저차원의 feature를 unsupervised로 학습하는 방식
  - Encoder : data로부터 저차원의 feature를 추출
  - Decoder : 저차원의 feature로부터 다시 data를 복원
  - Reconstruction training $\|x - \hat{x}\|^2$

Decoder가 복원 가능하다
= Data의 의미있는 요소들을
압축적으로 잘 담고 있음

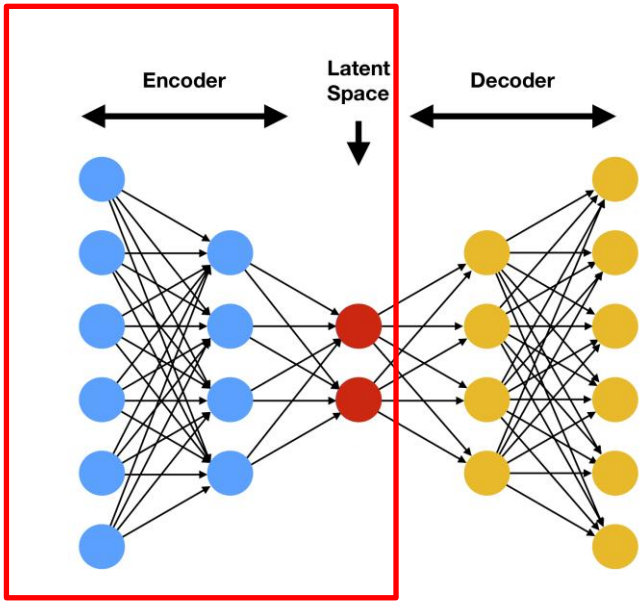
Data의 유용한 정보들을
함축적으로 잘 담고 있는
feature

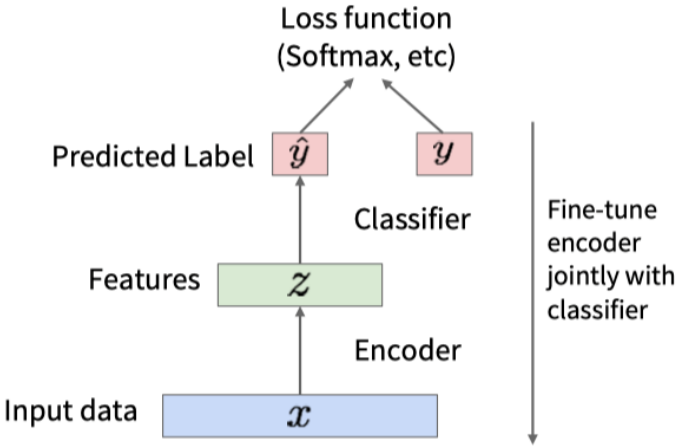


[1] https://www.compthree.com/blog/autoencoder/

Autoencoder

- AE
  - Data를 효율적이고 좋은 feature로 압축가능한 encoder를 다양하게 활용가능
    - Dimension reduction
    - Supervised model의 초기화

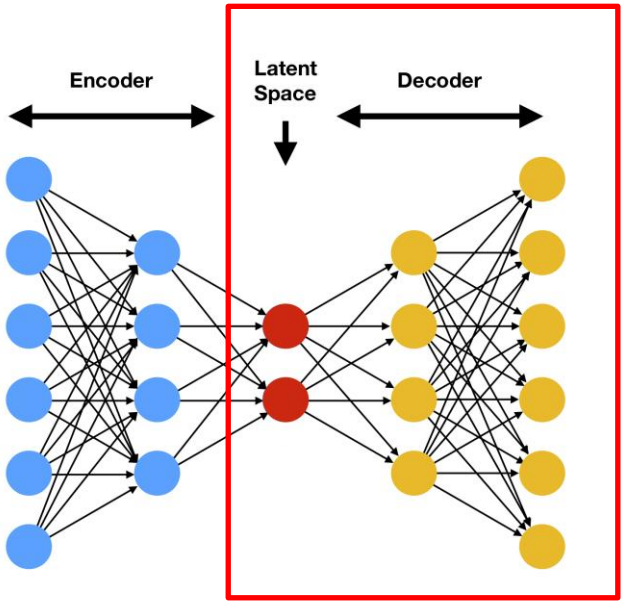


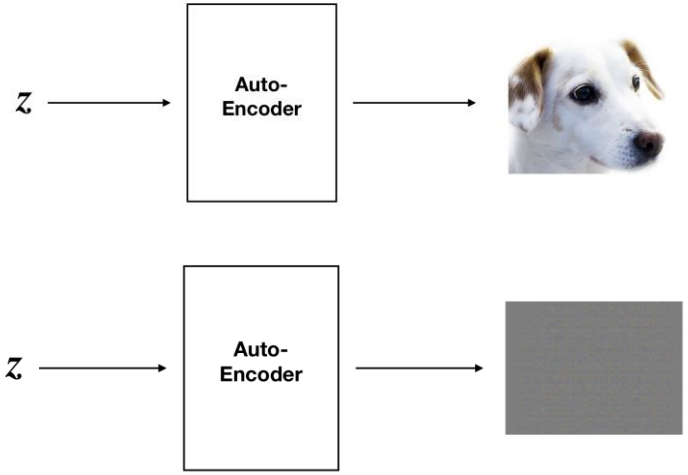


[1] https://www.compthree.com/blog/autoencoder/ [2] CS231n

Variational Autoencoder

- VAE
  - Autoencoder를 생성모델로 활용하기 위한 방법론
  - Latent space상의 적절한 z를 decoder의 input으로 사용하여 data를 만들어내고 싶음
  - 그러나 임의의 z를 decoder의 input으로 사용한다면 의미없는 sample을 만들어내게 됨
    - **Why? 임의의 z는 decoder가 학습된 latent space distribution에 있지 않기 때문**

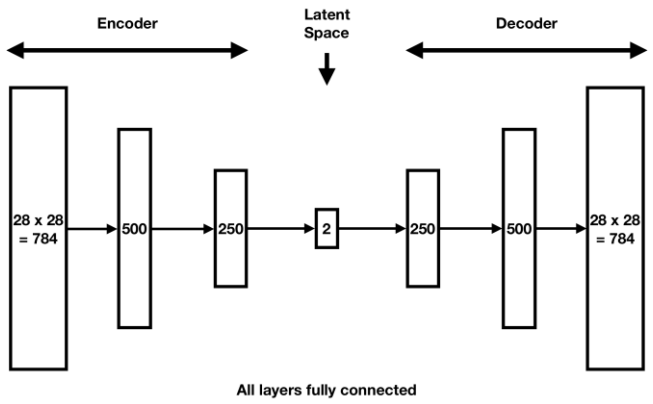


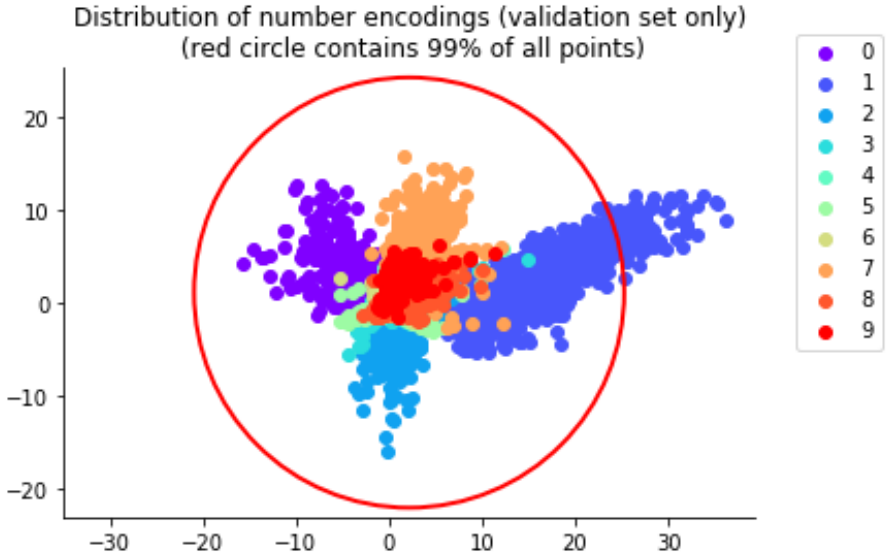


[1] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Autoencoder가 학습된 latent space는 굉장히 복잡하게 모델링 되어있음

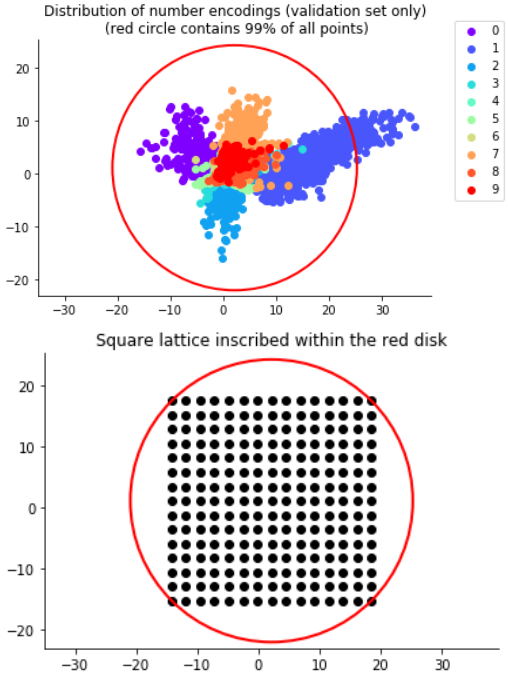


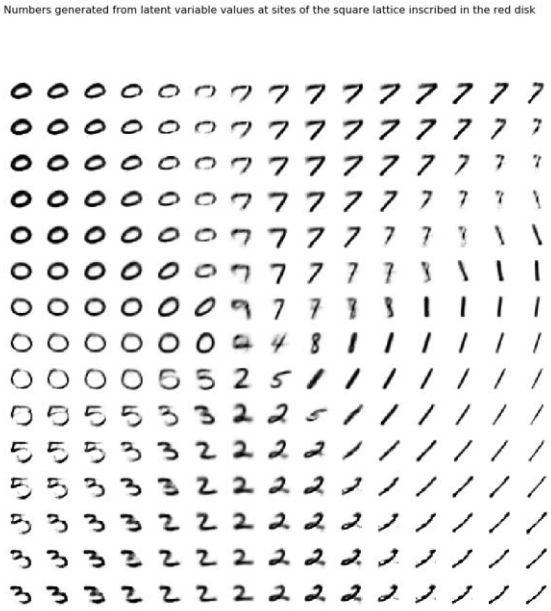


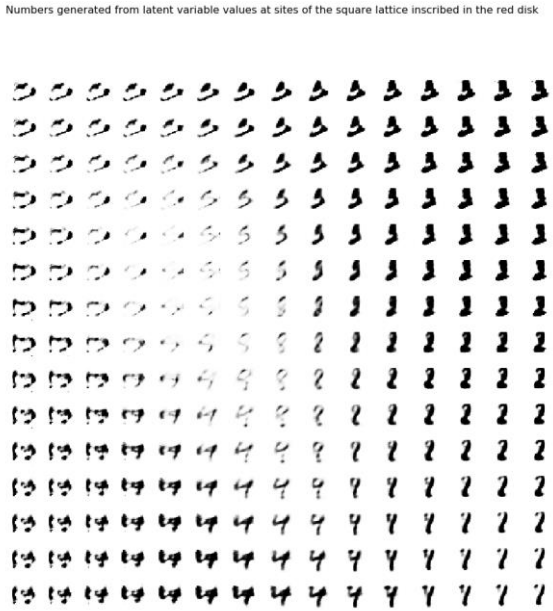
[1] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Autoencoder가 학습된 latent space는 굉장히 복잡하게 모델링 되어있음
  - 임의의 단순한 distribution에서 sampling한 z로부터 이미지가 생성되길 기대하는 것은 욕심!
    ~~모래사장에서 바늘찾기~~
  - 특히 latent space의 차원이 높아질 수록 자유도가 높아져 latent distribution이 복잡해짐



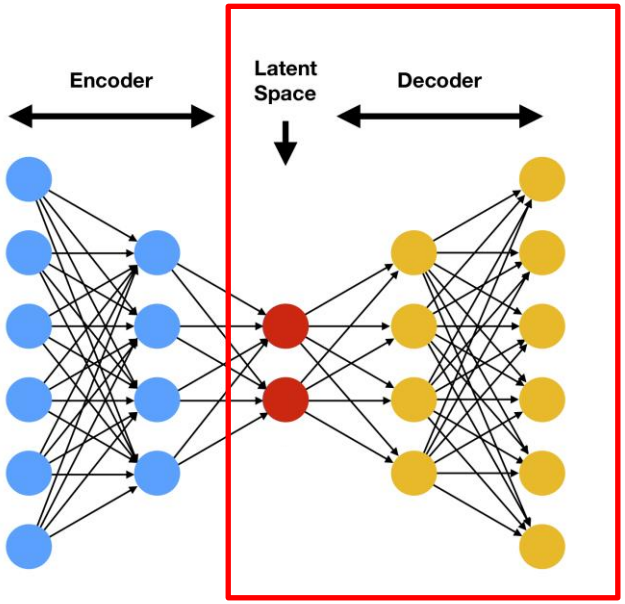


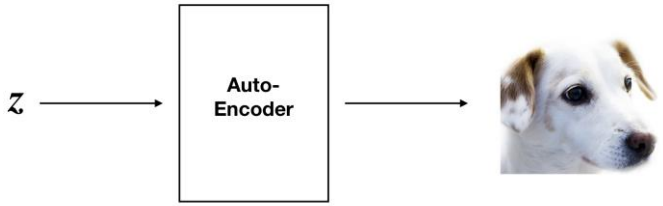


[1] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Autoencoder가 학습된 latent space는 굉장히 복잡하게 모델링 되어있음
  - Decoder가 복원할 수 있는 z (학습된 latent distribution의 z) 가 sampling이 가능해지도록 만든다면?
    - Decoder를 이용한 data 생성이 가능해짐!

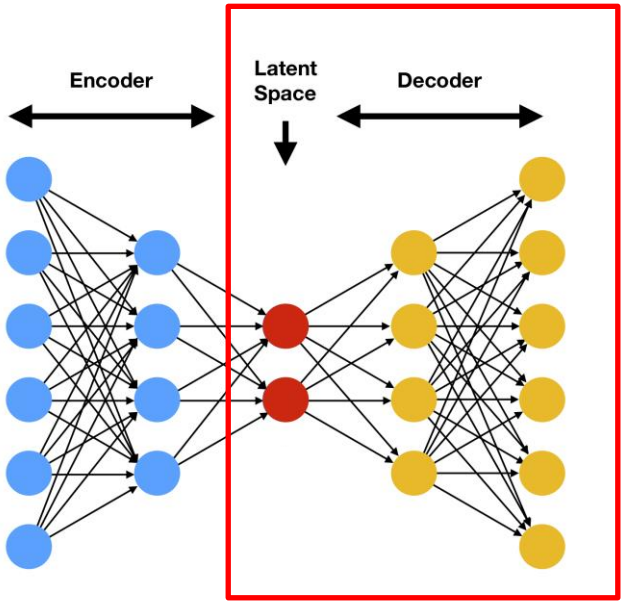


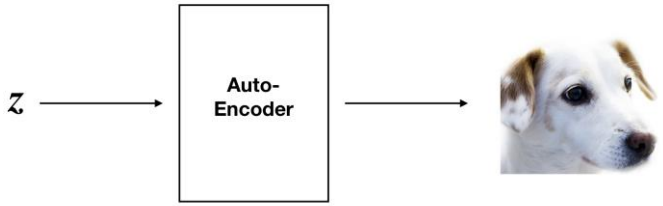


[1] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Autoencoder가 학습된 latent space는 굉장히 복잡하게 모델링 되어있음
  - Decoder가 복원할 수 있는 z (학습된 latent distribution의 z) 가 sampling이 가능해지도록 만든다면?
    - 학습된 latent의 distribution을 추가적인 생성모델로 생성하도록 학습
    - 처음부터 latent의 distribution이 sampling하기 쉬운 distribution이 되도록 제약을 걸어줌

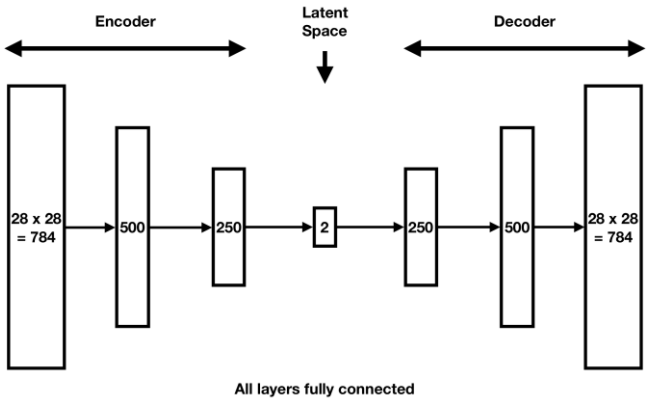


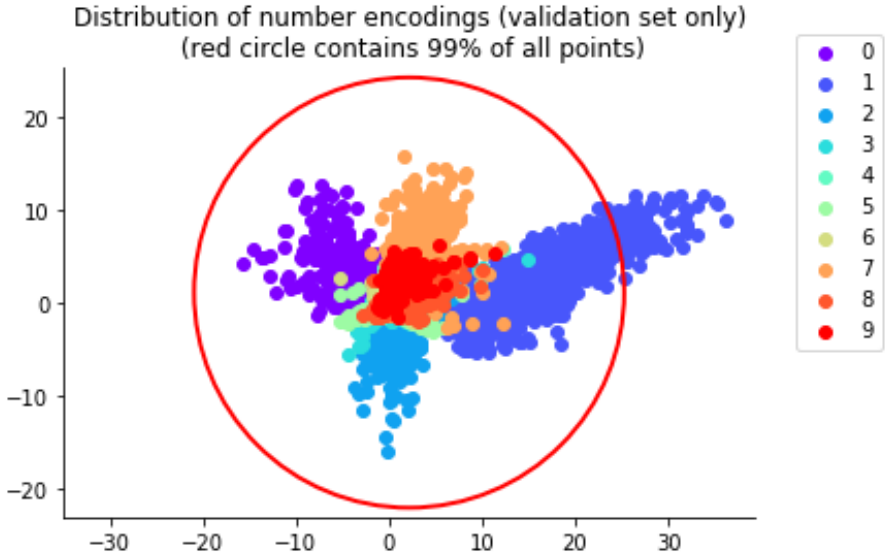


[1] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Autoencoder가 학습된 latent space는 굉장히 복잡하게 모델링 되어있음
  - Decoder가 복원할 수 있는 z (학습된 latent distribution의 z) 가 sampling이 가능해지도록 만든다면?
    - 학습된 latent의 distribution을 추가적인 생성모델로 생성하도록 학습
      → 생성모델로 latent를 생성한 다음 이를 decoder의 input으로 해서 최종적으로 data 생성
    - 처음부터 latent의 distribution이 sampling하기 쉬운 distribution이 되도록 제약을 걸어줌





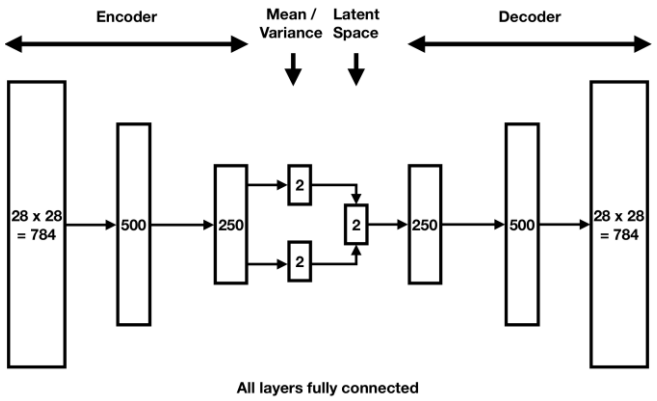
[1] https://www.compthree.com/blog/autoencoder/

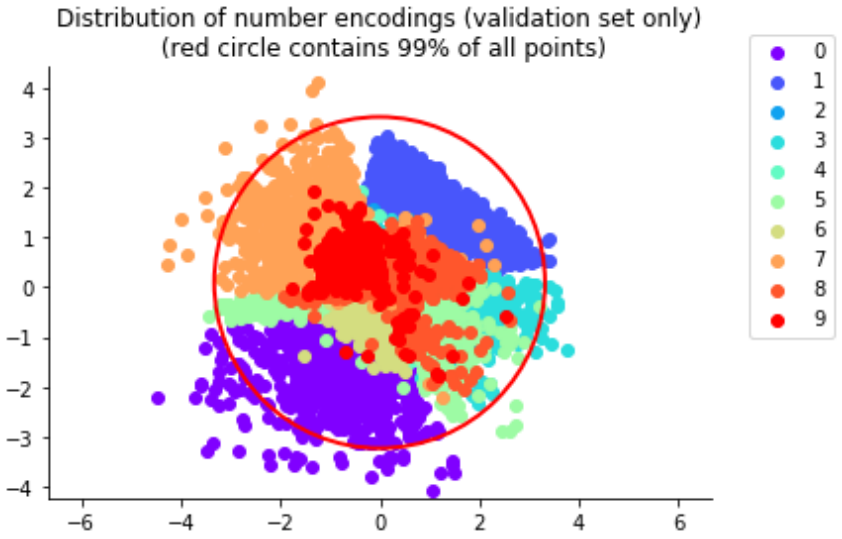
Variational Autoencoder

- VAE
  - Autoencoder가 학습된 latent space는 굉장히 복잡하게 모델링 되어있음
  - Decoder가 복원할 수 있는 z (학습된 latent distribution의 z) 가 sampling이 가능해지도록 만든다면?
    - 학습된 latent의 distribution을 추가적인 생성모델로 생성하도록 학습

      → 생성모델로 latent를 생성한 다음 이를 decoder의 input으로 해서 최종적으로 data 생성
    - 처음부터 latent의 distribution이 sampling하기 쉬운 distribution이 되도록 제약을 걸어줌

      → Sampling이 쉬운 distribution에서 뽑은 latent를 decoder의 input으로 하여 최종적으로 data 생성

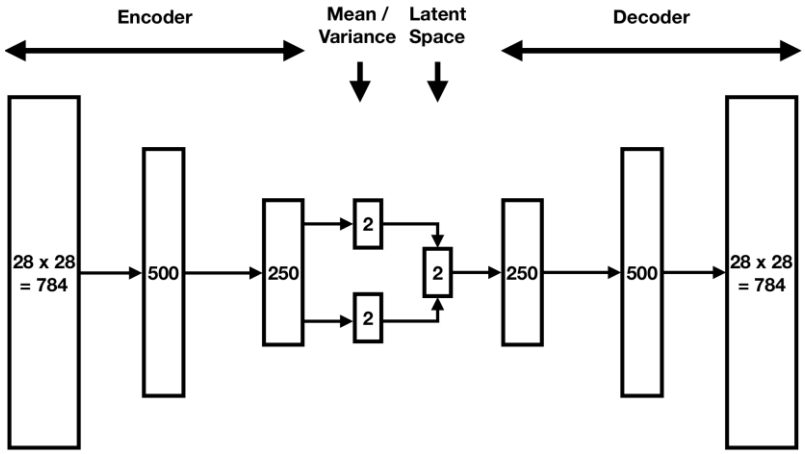




[1] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

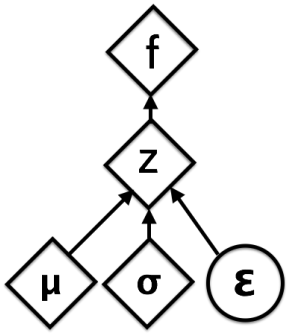
- VAE
  - Probabilistic autoencoder를 학습하면서 latent distribution이 sampling하기 쉬운 distribution이 되도록
  - Latent distribution
  - Reparameterization trick
  - Reconstruction loss
  - KL Divergence Regularization



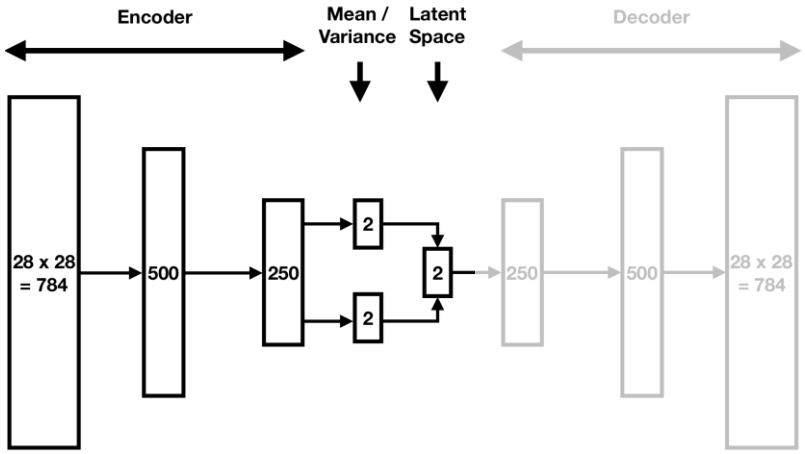
[1] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - **Probabilistic autoencoder**를 학습하면서 latent distribution이 sampling하기 쉬운 distribution이 되도록
  - Latent distribution : latent 대신 주어진 data에 대한 Gaussian distribution을 latent distribution으로 모델링
  - Reparameterization trick : distribution에서 sampling한 latent를 사용해도 gradient가 흐르도록
  - Reconstruction loss
  - KL Divergence Regularization



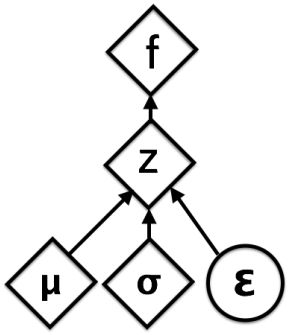
Reparametrized



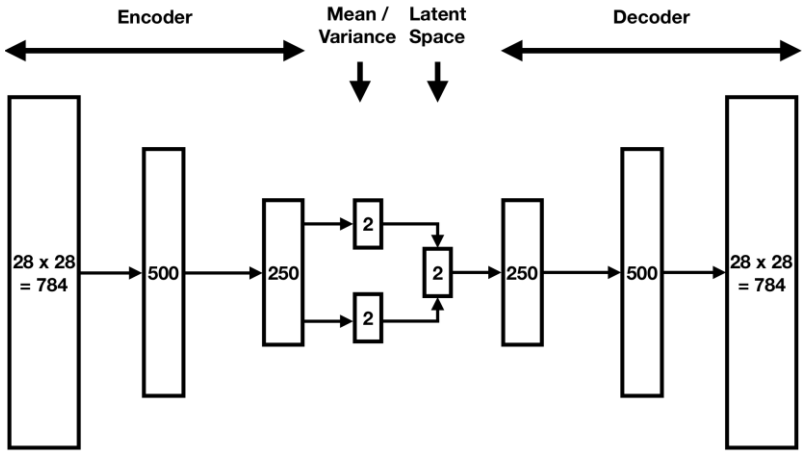
[1] https://jaan.io/what-is-variational-autoencoder-vae-tutorial/
[2] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Probabilistic autoencoder를 **학습하면서** latent distribution이 sampling하기 쉬운 distribution이 되도록
  - Latent distribution : latent 대신 주어진 data에 대한 Gaussian distribution을 latent distribution으로 모델링
  - Reparameterization trick : distribution에서 sampling한 latent를 사용해도 gradient가 흐르도록
  - Reconstruction loss $-E_{z\sim q(z|x)}[\log p(x \mid z)] = \|x - \hat{x}\|_2^2$
  - KL Divergence Regularization



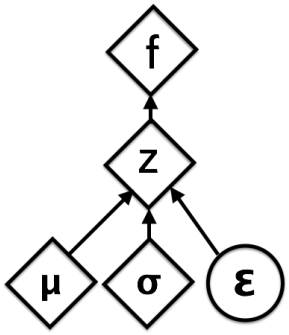
Reparametrized



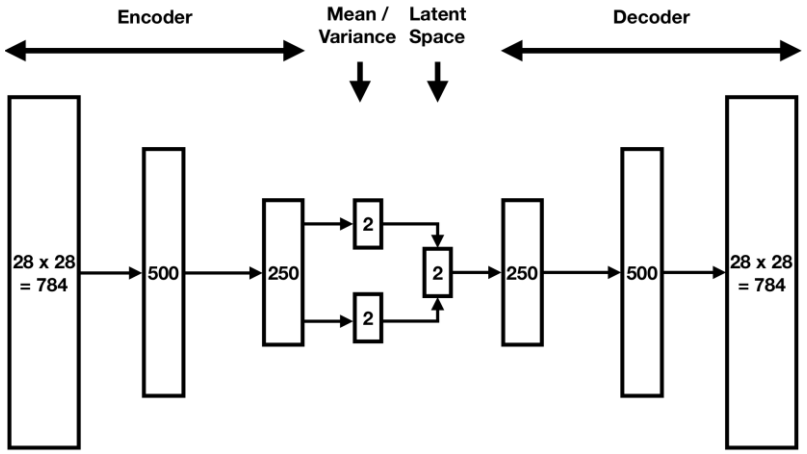
[1] https://jaan.io/what-is-variational-autoencoder-vae-tutorial/
[2] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Probabilistic autoencoder를 학습하면서 **latent distribution이 sampling하기 쉬운 distribution이 되도록**
  - Latent distribution : latent 대신 주어진 data에 대한 Gaussian distribution을 latent distribution으로 모델링
  - Reparameterization trick : distribution에서 sampling한 latent를 사용해도 gradient가 흐르도록
  - Reconstruction loss $-E_{z\sim q(z|x)}[\log p(x \mid z)] = \|x - \hat{x}\|_2^2$
  - KL Divergence Regularization $D_{KL}(q(z \mid x)\|p(z)) = \frac{1}{2}\sum_{i=1}\left(\sigma_i^2 + \mu_i^2 - \ln\left(\sigma_i^2\right) - 1\right)$



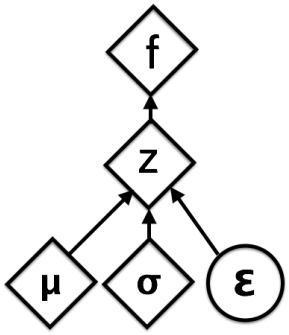
Reparametrized



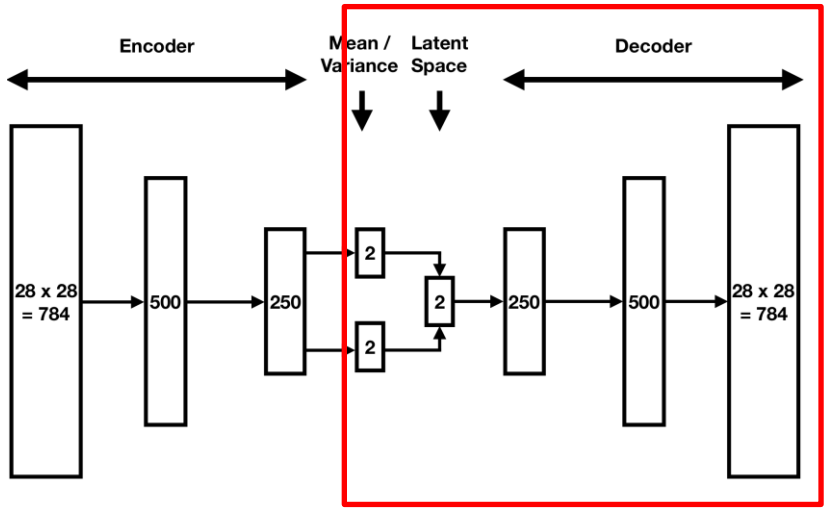
[1] https://ratsgo.github.io/generative%20model/2018/01/27/VAE/
[2] https://jaan.io/what-is-variational-autoencoder-vae-tutorial/
[3] https://www.compthree.com/blog/autoencoder/

Variational Autoencoder

- VAE
  - Autoencoder를 학습하면서 latent distribution이 sampling하기 쉬운 gaussian distribution이 되도록
  - Latent distribution : latent 대신 주어진 data에 대한 Gaussian distribution을 latent distribution으로 모델링
  - Reparameterization trick : distribution에서 sampling한 latent를 사용해도 gradient가 흐르도록
  - Reconstruction loss $-E_{z\sim q(z|x)}[\log p(x \mid z)] = \|x - \hat{x}\|_2^2$
  - KL Divergence Regularization $D_{KL}(q(z \mid x)\|p(z)) = \frac{1}{2}\sum_{i=1}\left(\sigma_i^2 + \mu_i^2 - \ln\left(\sigma_i^2\right) - 1\right)$



Reparametrized



Z from Gaussian distribution
= decoder가 학습된 영역이라
올바른 image 생성 가능

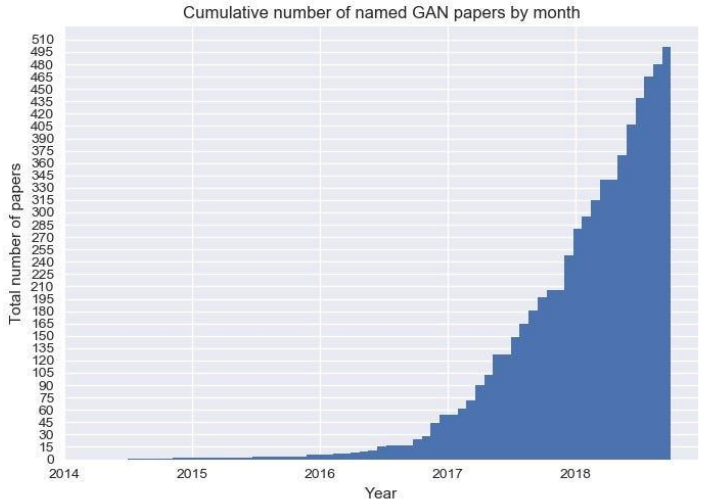
[1] https://jaan.io/what-is-variational-autoencoder-vae-tutorial/
[2] https://www.compthree.com/blog/autoencoder/

Autoencoder

- **[실습1] MNIST를 이용한 VAE 학습**
  - 1. VAE.ipynb
  - ToDo
    - VAE (encoder , decoder, reparameterization. trick) 구현
    - ELBO loss 구현 (Reconstruction loss 및 KL divergence loss)
    - VAE학습 및 생성 성능 확인
    - Latent 분석

Generative Adversarial Networks

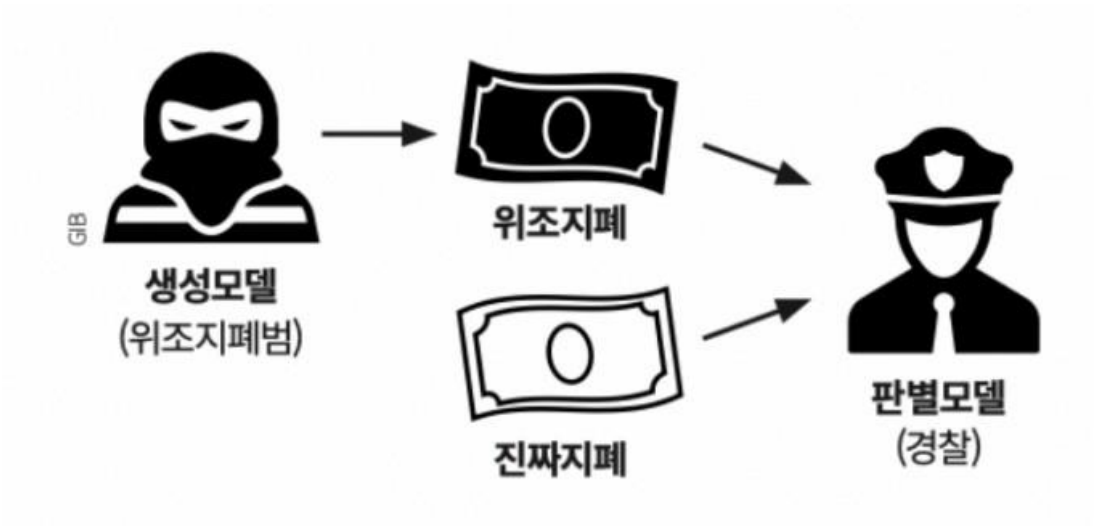
- GAN
  - 한 시대를 풍미한 생성모델의 과거의 최강자



[1] Ian Goodfellow twitter
[2] https://github.com/hindupuravinash/the-gan-zoo
[3] Analyzing and Improving the Image Quality of StyleGAN, Karras et al., 2020

Generative Adversarial Networks

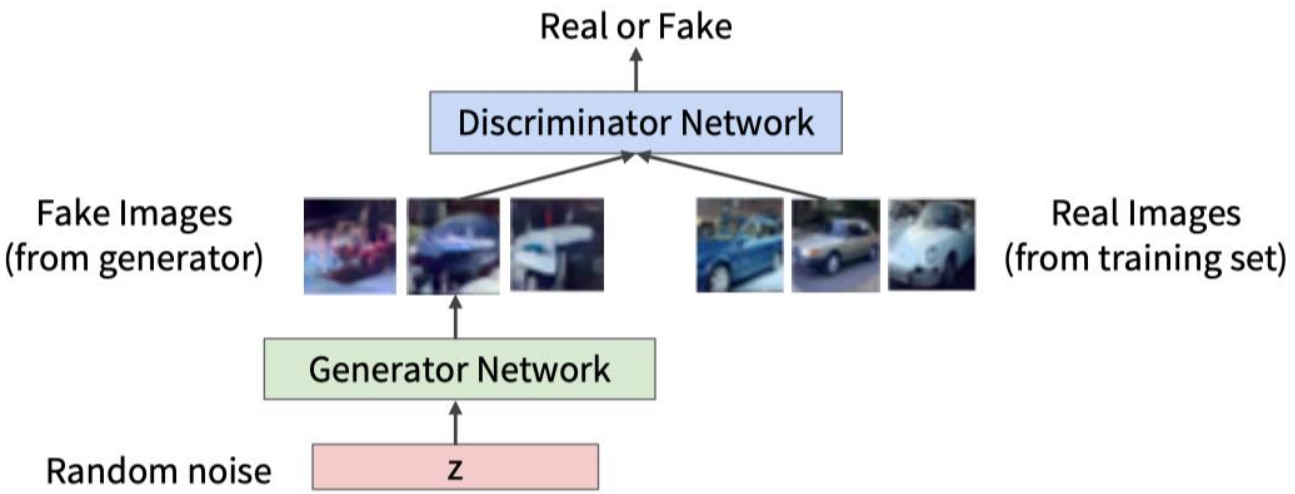
- GAN
  - 경찰과 위조지폐범의 경쟁에 비유
  - 위조지폐범: 위조지폐를 만들어냄
  - 경찰: 위조지폐범이 만든 지폐와 진짜 지폐를 가지고 비교해서 잡아냄
  - 위조지폐범: 경찰한테 안 걸리기 위해 더 좋은 위조지폐를 만들어냄



[1] https://m.dongascience.com/news.php?idx=54026

Generative Adversarial Networks

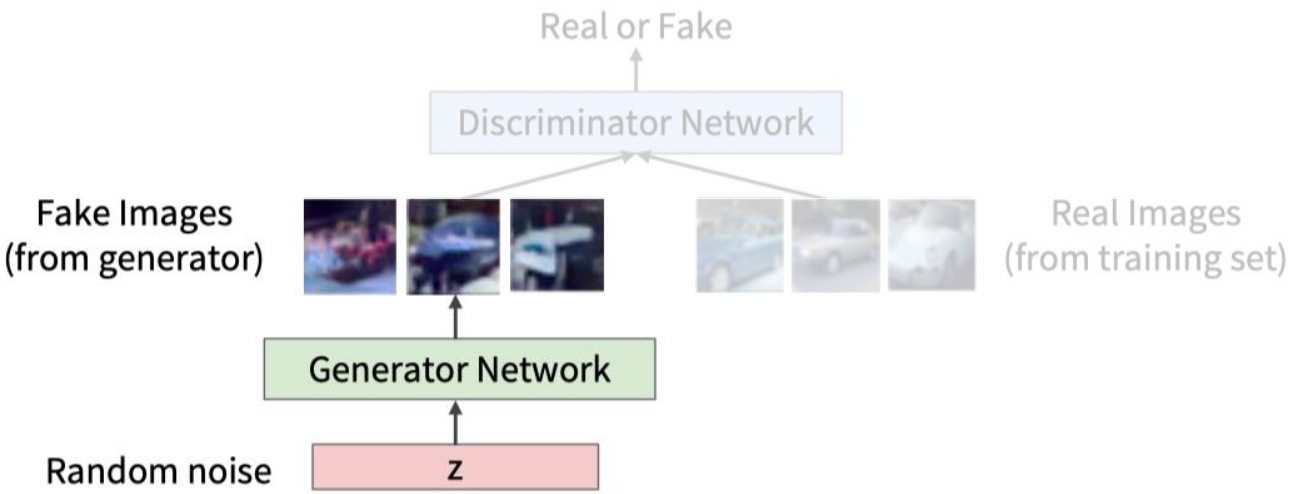
- GAN
  - 두 network의 적대적 학습을 통한 데이터 생성 학습
  - Generator
  - Discriminator
  - Adversarial Training



[1] CS231n: Deep Learning for Computer Vision, 2024

Generative Adversarial Networks

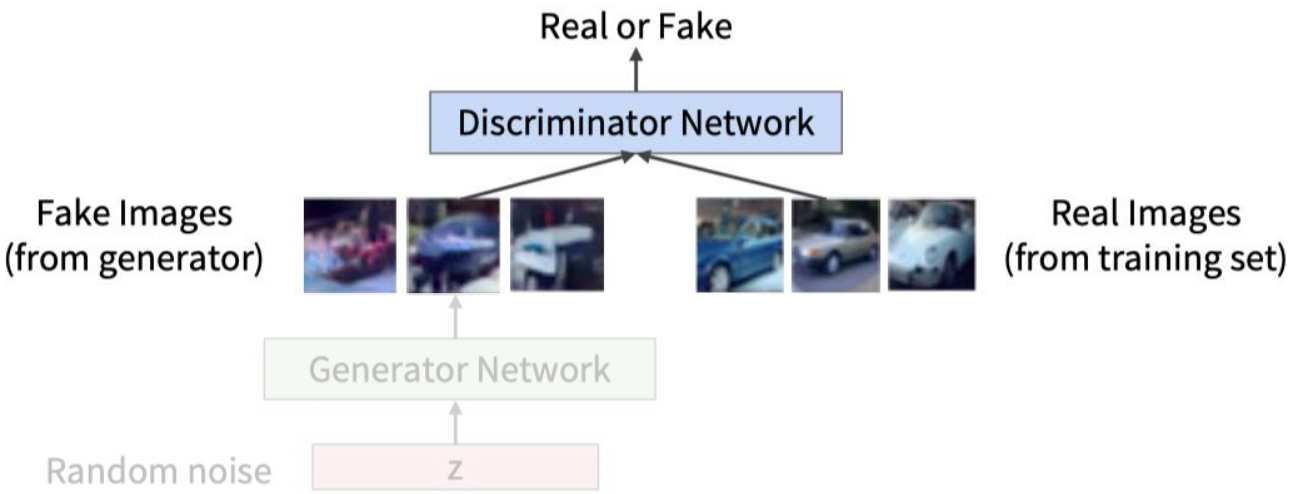
- GAN
  - **두 network**의 적대적 학습을 통한 데이터 생성 학습
  - Generator = random noise z로부터 sample을 직접 생성
  - Discriminator
  - Adversarial Training



[1] CS231n: Deep Learning for Computer Vision, 2024

Generative Adversarial Networks

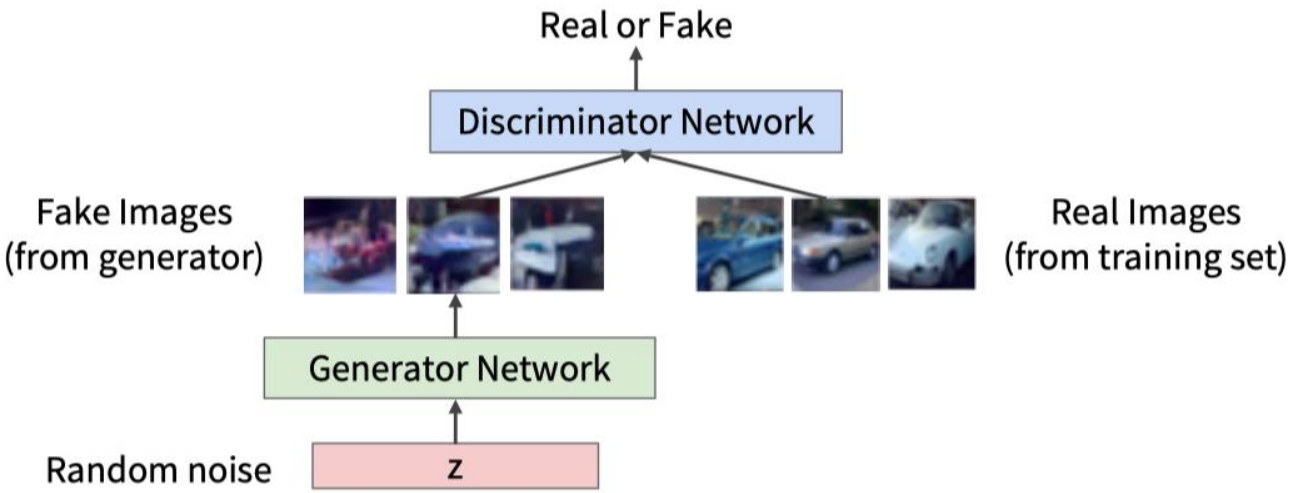
- GAN
  - **두 network**의 적대적 학습을 통한 데이터 생성 학습
  - Generator = random noise z로부터 sample을 직접 생성
  - Discriminator = generator가 생성한 sample과 real data sample을 구별
  - Adversarial Training



[1] CS231n: Deep Learning for Computer Vision, 2024

Generative Adversarial Networks

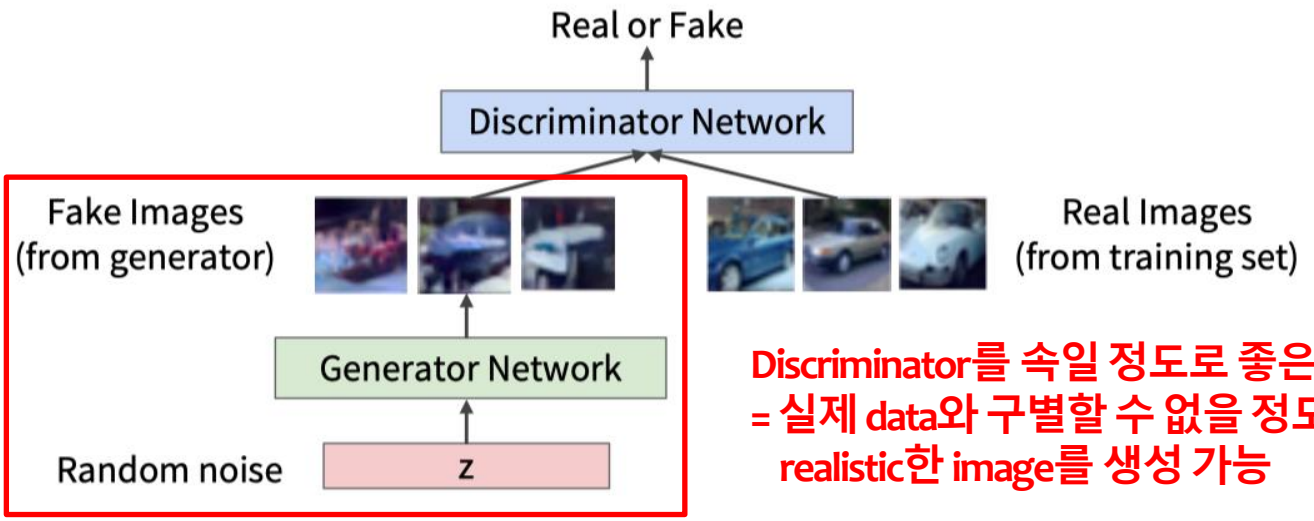
- GAN
  - 두 network의 **적대적 학습**을 통한 데이터 생성 학습
  - Generator = random noise z로부터 sample을 직접 생성
  - Discriminator = generator가 생성한 sample과 real data sample을 구별
  - Adversarial Training $\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log\left(1 - D_{\theta_d}\left(G_{\theta_g}(z)\right)\right)\right]$



[1] CS231n: Deep Learning for Computer Vision, 2024

Generative Adversarial Networks

- GAN
  - 두 network의 적대적 학습을 통한 데이터 생성 학습
  - Generator = random noise z로부터 sample을 직접 생성
  - Discriminator = generator가 생성한 sample과 real data sample을 구별
  - Adversarial Training $\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log\left(1 - D_{\theta_d}\left(G_{\theta_g}(z)\right)\right)\right]$

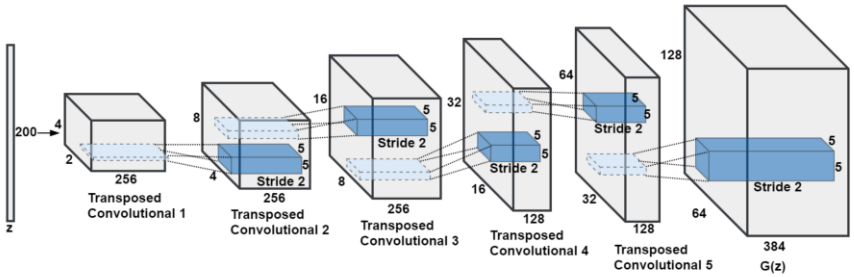


Discriminator를 속일 정도로 좋은 성능
= 실제 data와 구별할 수 없을 정도로
realistic한 image를 생성 가능

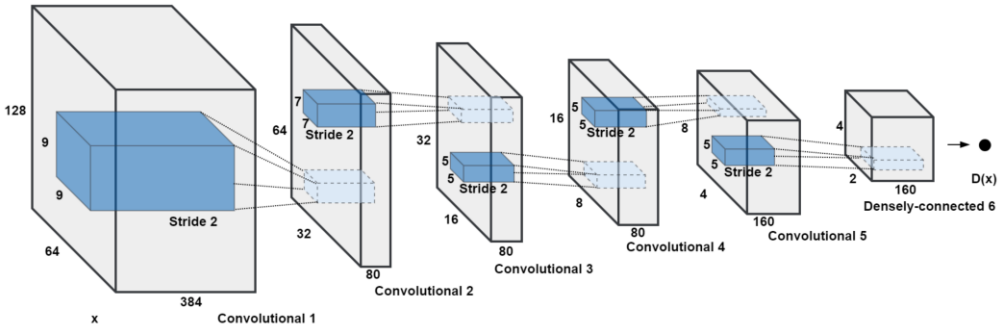
[1] CS231n: Deep Learning for Computer Vision, 2024

Deep Convolutional Generative Adversarial Networks

- DCGAN
  - 새로운 GAN architecture를 제안
  - Convolution layer를 이용해 두 network 구성



Generator



Discriminator

[1] https://medium.com/coinmonks/advancements-of-deep-learning-3-generative-adversarial-networks-gans-9d67010d656a

Deep Convolutional Generative Adversarial Networks

- **[실습2-1] DCGAN을 이용한 MNIST unconditional generation 학습**
  - 2-1. DCGAN-unconditional.ipynb
  - ToDo
    - Generator , Discriminator 구현
    - Adversarial loss 구현
    - Adversarial training 실행

Deep Convolutional Generative Adversarial Networks

- Class Conditional DCGAN
  - 기존의 DCGAN은 전체 dataset을 통째로 생성하도록 학습
  - Class라는 추가적인 정보를 통해 원하는 데이터를 생성하도록 조절

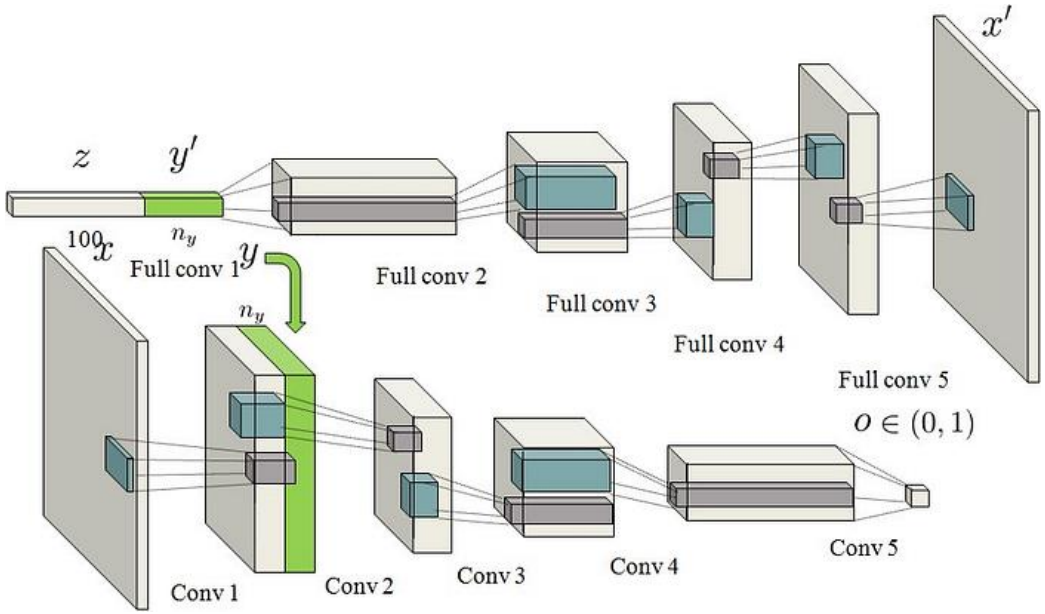
  - Generator = random noise z로부터 sample을 직접 생성
  - Discriminator = generator가 생성한 sample과 real data sample을 구별
  - Adversarial Training $\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z)\right)\right) \right]$

  ⬇ **Class label y 추가**

  - Generator = random noise z와 **주어진 class label y**로부터 sample을 직접 생성
  - Discriminator = generator가 생성한 sample과 real data sample을 **label y마다** 구별
  - Adversarial Training $\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x,y \sim p_{\text{data}}} \log D_{\theta_d}(x, y) + \mathbb{E}_{z \sim p(z), y' \sim p_y} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z, y'), y'\right)\right) \right]$

Deep Convolutional Generative Adversarial Networks

- Class Conditional DCGAN
  - Generator = random noise z와 **주어진 class label y**로부터 sample을 직접 생성
  - Discriminator = generator가 생성한 sample과 real data sample을 **label y마다** 구별
  - Adversarial Training $\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x,y\sim p_{\text{data}}} \log D_{\theta_d}(x,y) + \mathbb{E}_{z\sim p(z),y'\sim p_y} \log\left(1 - D_{\theta_d}\left(G_{\theta_g}(z,y'),y'\right)\right)\right]$



DSAIL Data Science & AI Laboratory

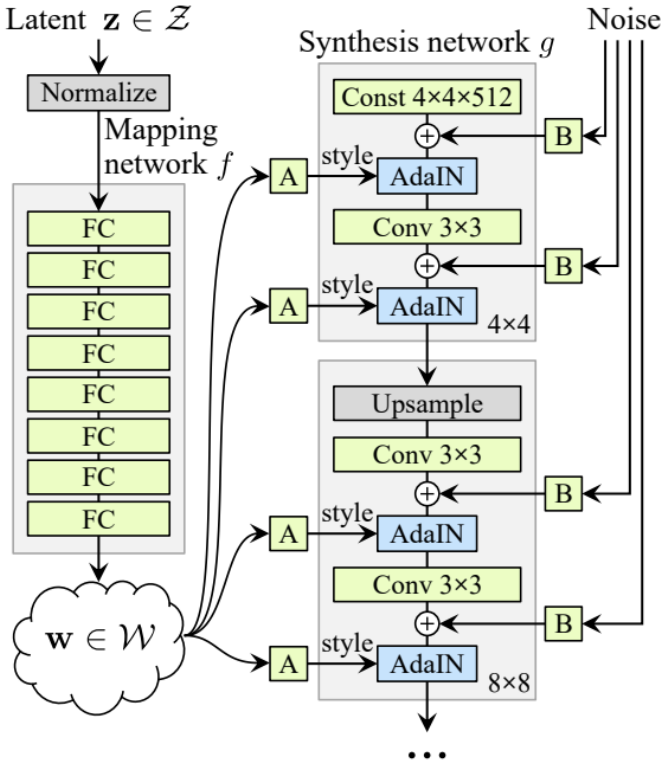
[1] https://medium.com/@sam.maddrellmander/conditional-dcgan-in-tensorflow-336f8b03b7b6

Deep Convolutional Generative Adversarial Networks

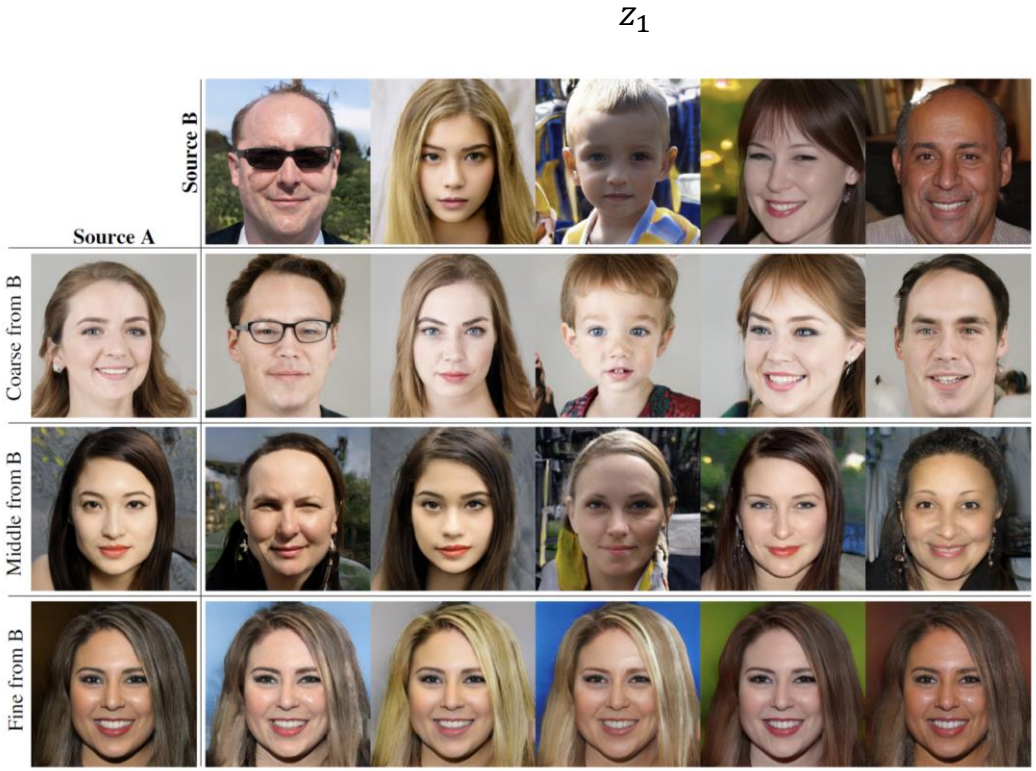
- **[실습2-2] DCGAN을 이용한 MNIST conditional generation 학습**
  - 2-2. DCGAN-conditional.ipynb
  - ToDo
    - Generator , Discriminator 구현 (conditional)
    - Adversarial loss 구현
    - Adversarial training 실행

A Style-Based Generative Adversarial Networks

- StyleGAN
  - Latent vector의 controllability를 극대화한 GAN 계열의 야심작



(b) Style-based generator



[1] A Style-Based Generator Architecture for Generative Adversarial Networks, Karras et al., 2019


Thank You!

Variational Autoencoder

- VAE
  - 궁극적으로 generation process안에서 training data들의 likelihood를 maximize하는 것
  - 이를 위해 autoencoder의 구조와 함께 복잡한 variational inference를 차용
  - ELBO를 maximize 하도록 학습하게 됨

$$
\begin{aligned}
\log p(\boldsymbol{x}) &= \mathbb{E}_{\mathbf{z}\sim q(\boldsymbol{z}|\boldsymbol{x})}[\log p(\boldsymbol{x})] \\
&= \mathbb{E}_{\mathbf{z}}\left[\log \frac{p(\boldsymbol{x}\mid\boldsymbol{z})p(\boldsymbol{z})}{p(\boldsymbol{z}\mid\boldsymbol{x})}\right] \\
&= \mathbb{E}_{\mathbf{z}}\left[\log \frac{p(\boldsymbol{x}\mid\boldsymbol{z})p(\boldsymbol{z})}{p(\boldsymbol{z}\mid\boldsymbol{x})}\frac{q(\boldsymbol{z}\mid\boldsymbol{x})}{q(\boldsymbol{z}\mid\boldsymbol{x})}\right] \\
&= \mathbb{E}_{\mathbf{z}}\left[\log p(\boldsymbol{x}\mid\boldsymbol{z})\right] - \mathbb{E}_{\mathbf{z}}\left[\log \frac{q(\boldsymbol{z}\mid\boldsymbol{x})}{p(\boldsymbol{z})}\right] + \mathbb{E}_{\mathbf{z}}\left[\log \frac{q(\boldsymbol{z}\mid\boldsymbol{x})}{p(\boldsymbol{z}\mid\boldsymbol{x})}\right] \\
&= \underbrace{\mathbb{E}_{\mathbf{z}}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}\mid\boldsymbol{z})\right] - D_{\mathrm{KL}}[q_{\boldsymbol{\phi}}(\boldsymbol{z}\mid\boldsymbol{x})\,||\,p(\boldsymbol{z})]}_{\text{ELBO: } \mathcal{L}(\boldsymbol{x},\boldsymbol{\theta},\boldsymbol{\phi})} + \underbrace{D_{\mathrm{KL}}[q_{\boldsymbol{\phi}}(\boldsymbol{z}\mid\boldsymbol{x})\,||\,p(\boldsymbol{z}\mid\boldsymbol{x})]}_{\geq 0 \text{ (intractable)}}
\end{aligned}
$$

[1] lecturen note “Variational Autoencoder” p.38

Autoencoder

- **[실습1] MNIST를 이용한 VAE 학습**
  - 1. VAE.ipynb
  - ToDo
    - VAE (encoder , decoder, reparameterization. trick) 구현
    - ELBO loss 구현 (Reconstruction loss 및 KL divergence loss)
    - VAE학습 및 생성 성능 확인
    - Latent 분석

Autoencoder

- **[실습1] MNIST를 이용한 VAE 학습**
  - 1. VAE.ipynb
  - ToDo
    - ELBO loss 구현 (Reconstruction loss 및 KL divergence loss)

$$\log p_\theta(x) \geq \mathbf{E}_z \left[\log p_\theta \left(x \mid z\right)\right] - D_{KL} \left(q_\phi \left(z \mid x\right) \| p_\theta(z)\right)$$

Autoencoder

- **[실습1] MNIST를 이용한 VAE 학습**
  - 1. VAE.ipynb
  - ToDo
    - ELBO loss 구현

$$\log p_\theta(x) \geq \mathbf{E}_z \left[\log p_\theta \left(x \mid z\right)\right] - D_{KL}\left(q_\phi \left(z \mid x\right) \| p_\theta(z)\right)$$

Reconstruction loss

Decoder가 복원한 data distribution 모델링에 따라

1. BCE Loss : data를 Bernoulli distribution으로 보는 경우 사용

$$-E_{z\sim q(z|x)}[\log p(x \mid z)] = -\sum_{i=1}^{N}\left[x_i \log\left(\hat{x}_i\right) + (1-x_i)\log\left(1-\hat{x}_i\right)\right]$$

2. L2 Loss : data를 Multi-variate Gaussian distribution으로 보는 경우 사용

$$-E_{z\sim q(z|x)}[\log p(x \mid z)] = \|x - \hat{x}\|_2^2$$

Autoencoder

- **[실습1] MNIST를 이용한 VAE 학습**
  - 1. VAE.ipynb
  - ToDo
    - ELBO loss 구현

$$\log p_\theta(x) \geq \mathbf{E}_z \left[\log p_\theta \left(x \mid z\right)\right] - D_{KL}\left(q_\phi \left(z \mid x\right) \| p_\theta(z)\right)$$

Regularization loss

Z의 prior distribution을 Normal distribution으로 설정

Encoder의 posterior distribution도 Gaussian distribution으로 설정

두 Gaussian distribution의 KL Divergence는 closed-form으로 계산 가능

$$D_{KL}(q(z \mid x) \| p(z)) = \frac{1}{2} \sum_{i=1} \left(\sigma_i^2 + \mu_i^2 - \ln\left(\sigma_i^2\right) - 1\right)$$

Deep Convolutional Generative Adversarial Networks

- **[실습2-1] DCGAN을 이용한 MNIST unconditional generation 학습**
  - 2-1. DCGAN-unconditional.ipynb
  - ToDo
    - Generator , Discriminator 구현
    - Adversarial loss 구현
    - Adversarial training 실행

Deep Convolutional Generative Adversarial Networks

- **[실습2-1] DCGAN을 이용한 MNIST unconditional generation 학습**
  - 2-1. DCGAN-unconditional.ipynb
  - ToDo
    - Adversarial loss 구현

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z)\right)\right) \right]$$

Discrimnator는 real과 fake를 구별하도록 학습
= real의 label을 1, fake의 label을 0으로 하는 binary classification 학습

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z)\right)\right) \right]$$

BCE loss와 형태 비슷

$$BCE = -\frac{1}{N} \sum_{i=0}^{N} y_i \cdot log(\hat{y}_i) + (1 - y_i) \cdot log(1 - \hat{y}_i)$$

Deep Convolutional Generative Adversarial Networks

- **[실습2-1] DCGAN을 이용한 MNIST unconditional generation 학습**
  - 2-1. DCGAN-unconditional.ipynb
  - ToDo
    - Adversarial loss 구현

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{\text{data}}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z)\right)\right) \right]$$

Generator는 Discriminator를 속이도록 학습
= fake의 label을 0이 나올 확률을 최소화

$$\min_{\theta_g} \left[\mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z)\right)\right) \right]$$

맥락은 비슷

= 대신 fake의 label을 1이 나올 확률을 최대화

$$\max_{\theta_g} \left[\mathbb{E}_{z \sim p(z), y' \sim p_y} \log \left(D_{\theta_d}\left(G_{\theta_g}(z, y'), y'\right)\right) \right]$$

$$BCE = -\frac{1}{N} \sum_{i=0}^{N} y_i \cdot log(\hat{y}_i) + (1 - y_i) \cdot log(1 - \hat{y}_i)$$

Deep Convolutional Generative Adversarial Networks

- **[실습2-2] DCGAN을 이용한 MNIST conditional generation 학습**
  - 2-2. DCGAN-conditional.ipynb
  - ToDo
    - Generator , Discriminator 구현 (conditional)
    - Adversarial loss 구현
    - Adversarial training 실행